Taşıma Ağı Optimizasyonu: İstatistiksel Parametre Tahmini, Modelleme ve Duyarlılık Analizi

Araçlar: Python (pandas, numpy, scipy, matplotlib/seaborn, PuLP veya Pyomo)

---

1. Amaç

Bu çalışmada bir firmanın depolardan müşteri bölgelerine yaptığı taşımayı en düşük maliyetle planlayan bir optimizasyon modeli kuracaksınız. Modelin **parametreleri (talep ve taşıma maliyeti) hazır verilmeyecek**; tarihsel veriden istatistiksel analizle siz tahmin edeceksiniz. Ardından sonuçları görselleştirecek ve bazı parametreler değiştiğinde sonucun nasıl etkilendiğini raporlayacaksınız.

2. Problem Tanımı

Bir lojistik firmasının 3 deposu (**Istanbul, Ankara, Izmir**) vardır ve 5 bölgeye (**Bursa, Konya, Antalya, Samsun, Gaziantep**) ürün gönderir. Her depoda aylık bir kapasite sınırı vardır. Amaç, her bölgenin aylık talebini karşılarken toplam aylık taşıma maliyetini minimize etmektir.

3. Veri Setinin Alınması

Komut `veri/` klasöründe bulunan üç dosya:

Dosya	İçerik
`talep_verisi.csv`	36 aylık tarihsel talep (ton), bölge bazında
`tasima_maliyeti.csv`	Depo–bölge güzergahlarında gerçekleşen sevkiyat kayıtları (TL/ton)
`kapasite.csv`	Depoların aylık kapasitesi (ton) – bu değerler **kesindir**, tahmin gerekmez

> *Dikkat: Gerçek verilerde olduğu gibi bu veride de **hatalar** olabilir. Veriye bakmadan analiz yapmayın.*

4. İstenenler

Bölüm A – Veri hazırlama ve istatistiksel analiz

1. Verileri `pandas` ile okuyun; `info()`, `describe()` ve eksik değer kontrolü yapın.
2. Eksik ve aykırı (mantıksız) gözlemleri tespit edin. Hangi yöntemi kullandığınızı (ör. medyan/IQR/kutu grafiği) ve bu gözlemleri nasıl ele aldığınızı gerekçesiyle yazın.
3. Her bölge için talebin ortalama, standart sapma, minimum ve maksimum değerlerini hesaplayın.
4. Her bölgenin ortalama talebi için **%95 güven aralığı** hesaplayın.
5. Talep verisinin normal dağılıma uygunluğunu bir yöntemle (histogram, Q-Q grafiği veya Shapiro-Wilk testi) değerlendirin; kısa yorum yazın.
6. Her güzergah (depo–bölge) için ortalama taşıma maliyetini hesaplayın ve 3x5 bir maliyet matrisi oluşturun.
7. **Model parametrelerini** belirleyin: bölge başına aylık talep (d_j) ve güzergah başına birim maliyet (c_ij). Neden ortalamayı seçtiğinizi (veya başka bir değer seçtiyseniz nedenini) açıklayın.

Bölüm B – Optimizasyon modeli

1. Matematiksel modeli yazın (karar değişkenleri, amaç fonksiyonu, kısıtlar):
   - x_ij: i. depodan j. bölgeye aylık taşınan miktar (ton)
   - Amaç: minimize Σ c_ij · x_ij
   - Kapasite: Σ_j x_ij ≤ K_i (her depo i için)
   - Talep: Σ_i x_ij ≥ d_j (her bölge j için)
   - x_ij ≥ 0
2. Modeli Python'da `PuLP` (veya `Pyomo`) ile kurun ve çözün.
3. Çözüm durumunu (Optimal / Infeasible), toplam maliyeti ve sevkiyat planını (depo x bölge tablosu) raporlayın.
4. Hangi depoların kapasitesinin sonuna kadar kullanıldığını belirleyin.

Bölüm C – Görselleştirme

En az **üç** grafik hazırlayın. Örnekler:

- Bölgelere göre ortalama talep ve güven aralıkları
- Sevkiyat planı (ısı haritası veya gruplu/yığılmış çubuk grafik)
- Depo kapasitesi ve kullanım oranı

Her grafikte başlık, eksen etiketleri (birimlerle) ve okunaklı gösterim olmalıdır. Her grafiğin altına bir iki cümlelik yorum ekleyin.

Bölüm D – Duyarlılık analizi

Aşağıdaki senaryoların her biri için modeli yeniden çözün ve sonuçları bir tabloda raporlayın (çözüm durumu, toplam maliyet, baz senaryoya göre değişim TL ve %):

No	Senaryo
1	Baz senaryo
2	Tüm depo kapasiteleri **%10 azalırsa**
3	Tüm depo kapasiteleri **%10 artarsa**
4	Tüm bölgelerin talebi **%5 azalırsa**
5	Tüm bölgelerin talebi **%5 artarsa**
6	Kapasite %10 azalıp **aynı anda** talep %5 artarsa

Ek olarak:

- Bu analizi bir fonksiyon (ör. `coz(kapasite_carpani, talep_carpani)`) yazarak tekrar eden kodla değil, **tek bir fonksiyonla** yapın.
- Sonuçları tek bir grafikle görselleştirin.
- Herhangi bir senaryoda model çözümsüz çıkarsa nedenini açıklayın.
- Sonuçlara dayanarak sorulara cevap verin: Maliyet en çok hangi parametreye duyarlıdır? Firma kapasite artırımı yapacaksa hangi depoyu seçmelidir? (İpucu: depoları tek tek %5–%10 değiştirip karşılaştırabilir, ya da `constraint.pi` ile gölge fiyatlara bakabilirsiniz.)

Bölüm E – Sunum

Kısa bir sunum hazırlayın: problem, yöntem, bulgular, grafikler, duyarlılık sonuçları ve yönetime 3–4 maddelik öneri. Kodunuzu (`.py` veya `.ipynb`) sunuma ekleyin.

5. Teslim

- Sunum
- Çalışan Python kodu (`.py` veya `.ipynb`)

6. İpuçları

- Kütüphane kurulumu: `pip install pandas numpy scipy matplotlib seaborn pulp`
- `df.groupby("bolge")["talep_ton"].mean()` bölge bazında ortalama alır.
- Güven aralığı için: `scipy.stats.t.ppf(0.975, n-1)` değerini kullanabilirsiniz.
- PuLP'ta kısıt eklerken `model += ifade <= sabit, "kisit_adi"` biçimini kullanın; kısıtlara isim vermek gölge fiyatlara erişmeyi kolaylaştırır.
- Önce baz senaryoyu çalıştırın, sonra senaryoları kodlayın.